

Contents

Implementation Plan: SAFS multi-fault SEAS smoke driver (mesh-shakedown)	1
Overview	1
Mesh tag confirmation	1
Constraints	2
Phase 1: Parameter struct + boundary config	3
Phase 2: Per-DOF friction parameter override	6
Phase 3: Smoke driver seas_safs_test_driver	8
Phase 4: Frontera verification run and acceptance	13
Differences vs BP5 setup — quick-reference table	16
Testing Strategy	17
Risk Assessment	18
What this plan deliberately leaves out	19

Implementation Plan: SAFS multi-fault SEAS smoke driver (mesh-shakedown)

Overview

Build a small, BP5-derived SEAS driver — `seas_safs_test_driver` — that exercises the SAFS multi-fault `.msh` end-to-end (DG elasticity solve → fault traction → rate-and-state friction → quasi-dynamic time stepping) for a few RK45 steps. The goal is mesh shakedown, not benchmark validation: confirm that the 6-fault embedded triangulation produced by the current production pipeline (`mesh/run_newset_step_by_step.sh` Step 6, with the 2026-05-02 R-001..R-006 + dihedral-aware orient + `check_mesh_tool_log` guards) does not blow up the solver and produces physically plausible per-step state (no NaN, slip rate within bounds, psi monotone, traction bounded), so we can move to a longer SAFS run with confidence.

The reference mesh for this driver is the cavity output of the production Step-6 build:

Metric	Value
Path	mesh/output/newset_6_all6_garnetfirst
gamma_min	6.51e-06
gamma_mean	0.850
Smallest tet edge (min_edge_m)	0.20 m
Mean fault tri edge	852 m
Mean in-tube tet edge	958 m (target res_f = 1000 m)
n_tets	1,099,696
Near-fault slivers (gamma < 0.05)	267 (0.024%)
Topology checks 5 / 12 / 13	all PASS

Mesh tag confirmation

The SAFS `.msh` (current production reference at `mesh/output/newset_6_all6_garnetfirst/output/safs_newset_6` produced 2026-05-02 by the Step-6 e2e run) already carries explicit physical groups for every box face plus the fault and the bulk volume:

```
$PhysicalNames
8
2 1  "xm"      ← x = -Lx vertical face
2 2  "xp"      ← x = +Lx vertical face
```

```

2 3  "yp"      ← y = +Ly vertical face (loading boundary, +X push)
2 4  "ym"      ← y = -Ly vertical face (loading boundary, -X push)
2 5  "ztop"    ← z = 0 free surface
2 6  "zbot"    ← z = -50 km lower boundary
2 100 "fault"  ← interior face physical group
3 10  "domain" ← bulk volume

```

Decision (option P): keep these geometric tag names; the driver performs the role mapping (yp/ym → Dirichlet loading, xm/xp/ztop/zbot → Natural) entirely in code via `BoundaryConfig`. We do not rename tags or re-run the mesh pipeline — the existing `.msh` is unchanged. Rationale: the mesh's tag numbers (1–6 + 100 + 10) are referenced by `mesh/validate_msh.py`, `mesh/run_*_cgal.sh`, the fault-provenance writer, and the expected-faults CLI plumbing. Renaming or adding tags forces a mesh + tooling change with no benefit beyond self-documentation.

Constraints

Hard constraints (from project conventions)

- Do NOT modify any file in `bp5/`, `bp1/`, `bp2/`, `domain/`, `fault/`, `solver/`, `friction/`, or `config/` — per `seas/CLAUDE.md` and the user-memory feedback `tpv102_bp5_no_shared_edit.md`. New code goes under `miniapps/seas/safs/code/`. If we need a duplicated utility, copy with the same signature; never edit the source.
- Reuse `ElasticityDomainOperator<ParMesh>`, `FaultGeometry<ParMesh>`, `RateStateFaultOperator<ParMesh,2>`, `SEASQuasiDynamicOperator`, and `DormandPrinceRK45` verbatim. The driver is a composition layer only.
- Use the `BoundaryConfig` (Phase-4) constructor of `ElasticityDomainOperator`, not the legacy `BCMode` constructor. SAFS mesh tags (1=xm, 2=xp, 3=yp, 4=ym, 5=ztop, 6=zbot, 100=fault) do not match the BP5 hardcoded numbering (1=Natural, 3=Fault, 5=Dirichlet), so the legacy constructor will misbind.
- Driver runs on Frontera; locally only build + unit tests. The current production SAFS mesh is ~1.1 M tets (see Overview); a single DG-elasticity solve at $p=1$ needs more memory than a typical laptop (mfem-dev build + MUMPS factorization). The shakedown driver therefore runs on Frontera in the development queue with 8 nodes, 400 cores total, 2 hour walltime. Locally we do NOT run the driver end-to-end; we only verify the code compiles (make `seas_safs_test_driver`) and that the existing seas unit-test suite still passes. The user-memory constraint `feedback_local_mpi_up_to_10.md` (≤ 10 MPI ranks locally) is preserved — local invocations stay under that bound and only exercise the build + unit-test surface.

Numerical / physics constraints

- Friction parameters: BP5 material ($\rho=2670$, $cs=3464$, $\nu=0.25$, $V_0=1e-6$, $f_0=0.6$, $b=0.03$, $\sigma_n=25$ MPa, $V_p=1e-9$).
- Rate-state direct effect a is uniform $\mathbf{a} = \mathbf{a}_0 = 0.004$ (all VW). No spatial variation, no a_{max} patch, no transition zone.
- Critical slip distance $L = L_0 = 0.14$ m uniform (no L_{nuc} patch).
- No nucleation seed: $V_{init} = (0, V_p)$ uniform in fault-local frame ($\text{tangent1}=\text{dip} \approx 0$, $\text{tangent2}=\text{strike} = V_p$). No V_{nuc} , no $\Delta\tau$. Initial state is identically the steady-state plate-rate configuration — by construction nothing should evolve to first order.
- Boundary loading: y-faces only. Dirichlet $u_X = \text{sgn}(Y) * V_p * t / 2$ on attrs 3 (yp) and 4 (ym). All other vertical attrs (1=xm, 2=xp) are Natural. Top/bottom (5, 6) Natural. This is a one-axis tectonic loading boundary set; xm/xp are free. We expect a small DC drift in displacement along x, which is fine for a few-step test.
- Time stepping: DormandPrinceRK45 with BP5 tolerances ($atol=1e-7$, $rtol=1e-50$, $dt_{min}=1e-6$, $dt_{max}=0.1$ yr). Initial $dt = \min(0.01 * L_0 / \max(V_{init}, V_p), dt_{max}) = 1.4e6 \text{ s} \approx 16$ days under default parameters ($L_0=0.14$, $V_{init}=V_p=1e-9$, so the $0.01 * L_0 / V$ term is $\sim 1.4e6$ s, well

below $dt_{max} \approx 3.156e6$ s; the cap is a safety net for future overrides that increase $L0/V_{init}$ past 36 days).

- `Mesh.MshFileVersion = 2.2` (matches what `safs.geo` writes).
- Polynomial order $p = 1$ (lowest order), BR2 DG method, MUMPS_BLR solver. We are not testing convergence; we want the cheapest correct configuration.

Interface constraints

- The driver must accept the `SAFS.msh` produced by the Step-6 production pipeline `mesh/run_newset_step_by_step START_FROM=6` with `ENABLE_DEDUP_COPLANAR=1`, `ENABLE_MMG3D_POSTPASS=1`, `MMG3D_MODE=optim_relax_fault`, `ENABLE_MMG3D_LOCAL_PATCH=0`, `ENABLE_LOCAL_CAVITY_RETET=1`, `LOCAL_CAVITY_GAMMA_THRESH=1e-3`. The output mesh is at `mesh/output/newset_6_all6_garnetfirst/output/safs_newset_6_cavity.msh` without any pre-processing.
- The driver must NOT depend on the per-fault provenance JSON (`fault_provenance.json`); fault tag 100 is the union of all 6 CFM faults and is treated as one fault interface for this test.
- Output: a single CSV (`safs_smoke.csv`) with one row per accepted RK45 step, columns: `step`, `t`, `dt`, `V_max`, `V_min`, `psi_max`, `psi_min`, `traction_max`, `slip_l2`, `n_dt_rejects`, `success`. No SCEC station outputs; this is not a benchmark run.

What we are explicitly NOT doing in this plan

- We are NOT validating physics (no SCEC reference, no fault-resolved ruptures expected — $a = a0$ uniform with $V_{init} = V_p$ is steady-state).
- We are NOT addressing the ~270 near-fault sliver tets; the test is the canary that tells us whether they matter.
- We are NOT computing per-fault diagnostics across the 6 CFM components separately. All fault DOFs are pooled.
- We are NOT solving for nucleation, rupture, or event statistics.

Phase 1: Parameter struct + boundary config

Goal

After this phase, a single header `safs_test_params.hpp` defines the SAFS test problem entirely (material constants, friction constants, geometry, boundary attribute mapping) without touching any existing config or fault code.

Files to Create

- `safs/code/safs_test_params.hpp` — header-only struct with constants and helper functions for uniform-VW, no-nucleation friction.
- `safs/code/safs_boundary_config.hpp` — header-only function that builds a `mfem::seas::BoundaryConfig` for SAFS mesh tags.

Files to Modify

- None.

Detailed Requirements

1.1 **SafsTestParams** struct File: `safs/code/safs_test_params.hpp`. Plain aggregate with the following exact field types and defaults (all in SI units; copy verbatim from `config/bp5_params.hpp` where applicable, but do not include `bp5_params.hpp` — duplicate the constants):

```

struct SafsTestParams {
    // Material – copied from BP5
    real_t rho      = 2670.0;
    real_t cs       = 3464.0;
    real_t nu       = 0.25;
    real_t mu()      const { return rho * cs * cs; }
    real_t lambda()  const { return 2.0 * nu * mu() / (1.0 - 2.0 * nu); }
    real_t eta()     const { return mu() / (2.0 * cs); }

    // Friction – uniform, all VW
    real_t V0       = 1.0e-6;
    real_t f0       = 0.6;
    real_t b        = 0.03;
    real_t a0       = 0.004;    // uniform a; no amax patch
    real_t L0       = 0.14;    // uniform Dc; no L_nuc patch

    // Stress
    real_t sigma_n  = 25.0e6;

    // Loading – same Vp as BP5; no nucleation seed
    real_t Vp       = 1.0e-9;
    real_t V_init   = 1.0e-9;  // == Vp; ensures equilibrium IC
    real_t V_zero   = 1.0e-20;

    // Boundary attrs (SAFS mesh; do not change without re-running safs.geo)
    int attr_xm     = 1;
    int attr_xp     = 2;
    int attr_yp     = 3;
    int attr_ym     = 4;
    int attr_ztop   = 5;
    int attr_zbot   = 6;
    int attr_fault  = 100;

    // Simulation
    static constexpr real_t seconds_per_year = 365.25 * 24.0 * 3600.0;
    real_t t_final = 1.0 * seconds_per_year;  // 1 year – short shakedown
    int n_steps_max = 100;                    // hard step cap

    // Steady-state psi at plate rate (constant – V_init = Vp uniform)
    real_t psi_init() const { return f0 + b * std::log(V0 / V_init); }

    // Initial pre-stress magnitude (uniform – equilibrium at V = V_init)
    // tau0 = sigma_n * a0 * asinh(V_init/(2 V0) * exp(psi_ss/a0)) + eta * V_init
    real_t tau0_scalar() const {
        const real_t psi_ss = psi_init();
        const real_t e      = std::exp(psi_ss / a0);
        return sigma_n * a0 * std::asinh((V_init / (2.0 * V0)) * e)
            + eta() * V_init;
    }

    // Validate (callable from driver init)
    void Validate() const {
        MFEM_VERIFY(a0 < b, "SAFS test requires a0 < b (VW)");
    }
}

```

```

    MFEM_VERIFY(L0 > 0.0, "SAFS test requires L0 > 0");
    MFEM_VERIFY(Vp > 0.0, "SAFS test requires Vp > 0");
    MFEM_VERIFY(sigma_n > 0.0, "SAFS test requires sigma_n > 0");
}

void Print(std::ostream& os = mfem::out) const; // implementation in .cpp
};

```

1.2 **MakeSafsBoundaryConfig** builder File: safs/code/safs_boundary_config.hpp. Exposes one function:

```
mfem::seas::BoundaryConfig MakeSafsBoundaryConfig(const SafsTestParams& p);
```

Body (header-only, inline) — this is the only divergence from BP5's MakeBP5DirichletFunc:

```

mfem::seas::BoundaryConfig MakeSafsBoundaryConfig(const SafsTestParams& p) {
    mfem::seas::BoundaryConfig cfg;
    cfg.fault_attr = p.attr_fault; // 100
    cfg.dirichlet_attrs = {p.attr_yp, p.attr_ym}; // {3, 4}
    cfg.natural_attrs = {p.attr_xm, p.attr_xp,
                        p.attr_ztop, p.attr_zbot}; // {1, 2, 5, 6}

    // Same loading expression as BP5's MakeBP5DirichletFunc – but only
    // applied to attrs 3 and 4 by virtue of dirichlet_attrs above.
    const real_t Vp = p.Vp;
    cfg.default_dirichlet_func =
        [Vp](const mfem::Vector& x, real_t t, mfem::Vector& u) {
            u.SetSize(3);
            u = 0.0;
            const real_t y = x(1);
            real_t Vh = Vp * t;
            if (y > 1000.0) { Vh *= 0.5; }
            else if (y < -1000.0) { Vh *= -0.5; }
            u(0) = Vh; // load along +x
        };
    return cfg;
}

```

The 1-km dead-band in the BP5 expression is irrelevant for SAFS because the y-face DOFs all sit at $|y| \geq 130$ km, but the expression is copied verbatim for source-level traceability.

Acceptance Criteria

- ☐ SafsTestParams::Validate() passes with default values.
- ☐ MakeSafsBoundaryConfig(p) returns a config whose dirichlet_attrs == {3, 4} and natural_attrs == {1, 2, 5, 6}.
- ☐ default_dirichlet_func evaluated at (x=0, y=2e5, z=0, t=1) returns $u = (Vp * 0.5, 0, 0)$; at (x=0, y=-2e5, z=0, t=1) returns $u = (-Vp * 0.5, 0, 0)$. Test as a hand-rolled assert in the driver.
- ☐ Header compiles in isolation against mfem.hpp and domain/boundary_config.hpp (no link errors).

Dependencies

- Depends on: nothing.

- Required by: Phase 2 (driver wires these in).

Phase 2: Per-DOF friction parameter override

Goal

After this phase, the driver can use `FaultGeometry<ParMesh>` constructed against `BP5Params` but with the `SAFS` uniform-`a`, uniform-`L`, no-nucleation overrides applied externally — without modifying `bp5_params.hpp` or `fault_geometry.hpp`.

Files to Create

- `safs/code/safs_fault_init.hpp` — small adapter that:
 1. Constructs a `BP5Params` from `SafsTestParams` (copies `BP5`-shaped fields into a temporary `BP5Params`, sets the geometric `VW` patch parameters such that the entire fault is inside the `VW` core).
 2. Provides a free-function `OverrideToUniformVW(BP5Params&, const SafsTestParams&)` that mutates the `BP5Params` before passing it to `FaultGeometry` so the per-DOF `a_arr`, `L_arr`, `V_init_vec`, `tau_pre` come out uniform.

Files to Modify

- None.

Detailed Requirements

2.1 Why we adapt `BP5Params` instead of writing a new `FaultGeometry` `fault/fault_geometry.hpp` only has constructors taking `BP2Params` or `BP5Params`. Adding a third would require modifying `fault_geometry.hpp` which is in the no-touch set. The cheapest workaround is: - Reuse `BP5Params` as the carrier struct. - Make the `BP5` spatial functions `a_of_x2_x3`, `IsNucleationZone`, `L_of_x2_x3`, `V_init_vec`, `tau0_vec` evaluate to the uniform values by setting `BP5Params` geometric fields such that every fault DOF is outside the `VW` patch's transition zone and outside the nucleation zone, while also setting `amax = a0` (so even when a DOF is computed in the `VS` zone, it still gets `a = a0` since `amax = a0`), and `L_nuc = L0`.

This is an exploit of existing `BP5` geometry knobs, not a code change.

```
// In safs_fault_init.hpp
inline void OverrideToUniformVW(mfem::seas::BP5Params& bp5,
                                const SafsTestParams& p) {
    // Material / friction / loading — copy from p
    bp5.rho    = p.rho;
    bp5.cs     = p.cs;
    bp5.nu     = p.nu;
    bp5.V0     = p.V0;
    bp5.f0     = p.f0;
    bp5.b      = p.b;
    bp5.a0     = p.a0;
    bp5.amax   = p.a0;           // KEY: a is uniform → set amax == a0
    bp5.L0     = p.L0;
    bp5.L_nuc  = p.L0;           // KEY: no nucleation patch → L uniform
    bp5.sigma_n = p.sigma_n;
```

```

bp5.Vp      = p.Vp;
bp5.V_init  = p.V_init;
bp5.V_nuc   = p.V_init;    // KEY: no nucleation seed → V uniform
bp5.delta_tau_factor = 0.0; // KEY: no overstress
bp5.smooth_nucleation = false;
bp5.nucleation_eps = 0.0;   // R-004: belt-and-suspenders so the
                           // strict numerical interval in
                           // IsNucleationZone is empty (default
                           // is 1e-3).

// Geometry fields – set so IsNucleationZone() always returns false.
// BP5's IsNucleationZone tests: (x3 in [hs+ht, hs+ht+H]) AND
//                               (x2 in [-l/2, -l/2 + w_nuc]).
// For SAFS, the fault sprawls over -135km < x2 < +123km and
// 0 < x3 < ~18km (depths in BP5 coords are positive). Setting
// w_nuc = 0 collapses the x2 strip to width 0, so the test is never
// true. Belt-and-suspenders: also push hs huge so the x3 test
// also fails.
bp5.w_nuc = 0.0;           // empty x2 strip
bp5.hs    = 1.0e7;         // 10000 km > any fault depth → x3 condition never true
bp5.ht    = 1.0;           // tiny but nonzero (avoid divide-by-zero in transition formula)
bp5.H     = 1.0;
bp5.l_vw  = 1.0e7;         // make VW core "everywhere" in x2
bp5.Wf    = 1.0e7;         // depth bound – well beyond fault
bp5.lf    = 1.0e7;

bp5.t_final = p.t_final;
}

```

2.2 OverrideToUniformVW signature and body The trick: - `IsNucleationZone(x2, x3)`: with `w_nuc = 0`, the strip $[-l/2, -l/2 + w_nuc] = [-l/2, -l/2]$ has zero width → no DOF satisfies $x2 \in$ this strip → always false. (BP5 uses $\leq$ in the condition; an $x2$ exactly equal to $-l/2$ would technically match, but we offset by `nucleation_eps = 1e-3` so the strict numerical interval is empty.) Set `bp5.nucleation_eps = 0.0` to be safe. - `a_of_x2_x3(x2, x3)`: with `amax = a0 = 0.004`, every branch returns the same value. Confirm by reading `bp5_params.hpp:192–213`: - VW core branch: return `a0` ✓ - VS zones branch: return `amax = a0` ✓ - Transition branch: return `a0 + r*(amax-a0) = a0 + 0 = a0` ✓ - `L_of_x2_x3(x2, x3)`: returns `L_nuc` if in nucleation zone, else `L0`. With `IsNucleationZone` always false → `L0`, and `L_nuc = L0` so it doesn't matter. - `V_init_vec(x2, x3, V)`: returns `(V_zero, V_init)` if NOT in nucleation, `(V_zero, V_nuc)` if in. Both `V_init` and `V_nuc` are set to `p.V_init` → uniform `(V_zero, V_init)` everywhere. - `tau0_vec(x2, x3, tau)`: with uniform `Vi`, `a`, and `delta_tau_factor=0`, returns the same value at every DOF.

Add a unit test (Phase 4) that checks `bp5.a_of_x2_x3(0, 0) == p.a0`, `bp5.a_of_x2_x3(1e6, 1e6) == p.a0`, and `bp5.IsNucleationZone(*, *) == false` for several sample points. If the existing BP5 `a_of_x2_x3` ever changes, this test catches the regression.

2.3 Edge cases

- `BP5Params::Validate()` checks `V_nuc >= V_init` (true: equal), `L_nuc < L0` (FALSE: equal). Important: do NOT call `bp5.Validate()` after our override. The driver should call `p.Validate()` (the SAFS one) instead.
- `BP5Params::Validate()` also checks `amax > b`. With `amax = a0 = 0.004` and `b = 0.03`, this is FALSE. So skipping `bp5.Validate()` is required.

Acceptance Criteria

- After `OverrideToUniformVW(bp5, p)`, calling `bp5.a_of_x2_x3(x2, x3)` with sample points $(0, 0)$, $(50000, 5000)$, $(-100000, 18000)$ returns `p.a0` exactly.
- `bp5.IsNucleationZone(x2, x3)` returns `false` for the same sample points.
- `bp5.L_of_x2_x3(x2, x3) == p.L0` for the same sample points.
- `bp5.tau0_vec(0, 0, tau)` and `bp5.tau0_vec(1e5, 5000, tau)` return the same `tau[0]`, `tau[1]` (within $1e-12$ relative).

Dependencies

- Depends on: Phase 1 (`SafsTestParams`).
 - Required by: Phase 3 (driver).
-

Phase 3: Smoke driver `seas_safs_test_driver`

Goal

Compile a parallel driver that loads the SAFS `.msh`, runs `N` RK45 steps, and writes a CSV with per-step diagnostics.

Files to Create

- `safs/code/seas_safs_test_driver.cpp` — main entry point.
- `safs/code/CMakeLists.txt` (or extend the existing `seas` Makefile) — add the new target. Actual mechanism: piggyback on the existing `miniapps/seas/Makefile`. Add a target `seas_safs_test_driver` that builds `safs/code/seas_safs_test_driver.cpp` linked against the same libraries as `seas_bp5_full`.

Files to Modify

- `miniapps/seas/Makefile` — add ONE new target stanza (`seas_safs_test_driver`) with object/source listings mirroring `seas_bp5_full`. Do not change any rule or variable used by the existing targets. This is the only no-touch boundary we cross; the addition is strictly additive.

Detailed Requirements

3.1 CLI signature

```
seas_safs_test_driver
  --mesh FILE           (required) path to SAFS .msh
  --output-csv FILE     (default: safs_smoke.csv)
  --order P             (default: 1)
  --n-steps N           (default: 100, hard cap on RK45 accepted steps)
  --dt-init DT          (default: -1, meaning use  $0.01 * L0 / Vp$  clamped to dt_max)
  --solver mumps|cg     (default: mumps)
  --print-every K       (default: 10)
  --dump-bdr-vtk        (optional flag)
```

`--mesh` is REQUIRED. No inline-mesh path — SAFS is the only target.


```

int main(int argc, char* argv[]) {
    Mpi::Init(argc, argv);
    Hypre::Init();

    // 1. Parse args -> SafsTestParams params, n_steps, csv_path, ...
    SafsTestParams params;
    params.Validate();
    if (rank == 0) params.Print();

    // 2. Load mesh (parallel).
    Mesh serial_mesh(mesh_path, 1, 1);
    serial_mesh.SetCurvature(1); // SAFS uses linear elements
    ParMesh pmesh(MPI_COMM_WORLD, serial_mesh);
    serial_mesh.Clear();

    // 3. Sanity checks on attrs:
    // - bdr_attributes contains at least {1,2,3,4,5,6}
    // - attributes contains {10}
    // - presence of triangle physical group 100 verified later via fault DOF count
    AssertSafsAttrs(pmesh, params);
    if (dump_bdr_vtk) pmesh.PrintBdrVTU(out_dir + "/safs_bdr");

    // 4. Build BoundaryConfig.
    BoundaryConfig bdr = MakeSafsBoundaryConfig(params);

    // 5. Build LinearElastic + ElasticityDomainOperator.
    LinearElastic elastic(params.lambda(), params.mu());
    ElasticityDomainOperator<ParMesh> domain(
        pmesh, order, elastic,
        params.Vp,
        /*Wf=*/0.0, // unused when no nucleation patch
        /*lf=*/0.0,
        bdr,
        DGMMethod::BR2,
        SolverType::MUMPS_BLR);
    domain.SetCheckResidual(true);

    // 6. Build BP5Params with SAFS overrides + FaultGeometry.
    BP5Params bp5;
    OverrideToUniformVW(bp5, params);
    FaultGeometry<ParMesh> fault_geom(domain, bp5, &mpi);
    if (fault_geom.NumGlobalFaultDOFs() == 0) {
        if (rank == 0) std::cerr
            << "ERROR: SAFS mesh has no fault DOFs (tag 100 missing?)\n";
        return 2;
    }

    // 7. Construct friction + state evolution + fault op.
    DieterichRuinaFriction::Constants fc;
    fc.V0 = params.V0; fc.f0 = params.f0; fc.b = params.b; fc.Dc = params.L0;
    DieterichRuinaFriction friction(fc);
    AgingLawPsi aging(params.b, params.V0, params.f0);
    RateStateFaultOperator<ParMesh, 2> fault_op(

```

```

    &fault_geom, &friction, &aging, bp5, &mpi);
// Use Tandem-style equilibrium psi (default; matches default behaviour).

// 8. SEAS quasi-dynamic operator.
PBP5SEASOp seas_op(&domain, &fault_op, &mpi);
seas_op.SetElasticSigmaN(true); // BP5 default

// 9. Initial state from operator.
// R-001: state must be pre-sized; SetInitialCondition opens with
// MFEM_VERIFY(state.Size() == fault_>StateSize(), ...). Mirrors
// drivers/seas_driver.cpp:364.
Vector state(fault_op.StateSize());
seas_op.SetInitialCondition(state);

// 10. Time stepper.
DormandPrinceRK45 ode_solver;
ode_solver.SetMPIContext(&mpi);
ode_solver.SetAbsTol(1e-7);
ode_solver.SetRelTol(1e-50);
ode_solver.SetDtMin(1e-6);
ode_solver.SetDtMax(0.1 * SafsTestParams::seconds_per_year);
// R-006: dt_init = 0.01 * L_min / V_max_init per CLAUDE.md.
// For this plan V_init == V_nuc == Vp, so all three are
// equivalent – but write the safe form so future overrides that
// raise V_init or V_nuc do NOT silently produce an over-large
// dt that triggers RK45 stage amplification (debug v7).
const real_t V_init_max = std::max(params.V_init, params.Vp);
real_t dt_init = (cli_dt_init > 0.0) ? cli_dt_init :
    std::min(0.01 * params.L0 / V_init_max,
             0.1 * SafsTestParams::seconds_per_year);
ode_solver.SetDt(dt_init);
ode_solver.SetStatePerNode(3); // [slip_dip, slip_strike, psi]
ode_solver.Init(seas_op);

// 11. Open CSV; write header.
OpenCsv(csv_path, /*header=*/...);

// 12. Time-step loop.
// R-002 + R-008: DormandPrinceRK45::Step takes 4 args
// (op, state, t, dt) and returns bool (true = accepted, false =
// rejected). No public getter for rejection count exists, so the
// driver maintains its own counter from the boolean return.
// Mirrors drivers/seas_driver.cpp:587–590.
real_t t = 0.0;
int step = 0;
int n_dt_rejects = 0;
while (step < n_steps_max) {
    // Inner loop: re-attempt rejected steps; each rejection
    // increments the driver-local counter.
    bool accepted = false;
    real_t dt = 0.0;
    while (!accepted) {
        accepted = ode_solver.Step(seas_op, state, t, dt);
    }
}

```

```

    if (!accepted) { ++n_dt_rejects; }
}
++step;

// R-009: Diagnostics on `state`. Only `GetMaxSlipRate` and
// `GetTraction` exist as direct getters on seas_op; the rest
// (V_min, psi_max, psi_min, traction_max, slip_l2) require
// explicit extraction from the state vector and MPI reduction.
const real_t V_max = seas_op.GetMaxSlipRate();
const Vector& trac = seas_op.GetTraction();

constexpr int spn = RateStateFaultOperator<ParMesh, 2>::StatePerNode;
const int n_local = state.Size() / spn;
real_t local_V_min = std::numeric_limits<real_t>::infinity();
real_t local_psi_min = std::numeric_limits<real_t>::infinity();
real_t local_psi_max = -std::numeric_limits<real_t>::infinity();
for (int i = 0; i < n_local; ++i) {
    const real_t v0 = state(i*spn + 0);
    const real_t v1 = state(i*spn + 1);
    const real_t psi = state(i*spn + 2);
    const real_t v_norm = std::hypot(v0, v1);
    local_V_min = std::min(local_V_min, v_norm);
    local_psi_min = std::min(local_psi_min, psi);
    local_psi_max = std::max(local_psi_max, psi);
}
real_t local_trac_max = 0.0;
for (int k = 0; k < trac.Size(); ++k) {
    local_trac_max = std::max(local_trac_max, std::abs(trac(k)));
}
const real_t V_min = mpi.GlobalReduceMin(local_V_min);
const real_t psi_min = mpi.GlobalReduceMin(local_psi_min);
const real_t psi_max = mpi.GlobalReduceMax(local_psi_max);
const real_t traction_max = mpi.GlobalReduceMax(local_trac_max);

Vector slip;
fault_op.GetSlip(state, slip);
const real_t local_slip_sq = slip * slip; // sum of squares
const real_t slip_l2 = std::sqrt(mpi.GlobalSum(local_slip_sq));

// Write one CSV row: step, t, dt, V_max, V_min, psi_max,
// psi_min, traction_max, slip_l2, n_dt_rejects, success.

// Hard sanity checks (abort on fail):
if (!std::isfinite(V_max) || !std::isfinite(psi_max)) {
    if (rank == 0) std::cerr
        << "ABORT: NaN/Inf at step " << step << "\n";
    return 3;
}
if (V_max > 10.0) // physical upper bound for SEAS slip rate
{
    if (rank == 0) std::cerr
        << "ABORT: V_max = " << V_max << " > 10 m/s at step "
        << step << "\n";
}

```

```

        return 4;
    }
    if (psi_min < 0.0 || psi_max > 5.0) {
        if (rank == 0) std::cerr
            << "ABORT: psi out of [0, 5] at step " << step << "\n";
        return 5;
    }

    // Periodic console summary.
    if (rank == 0 && (step % print_every) == 0) {
        std::cout << "step " << step << " t=" << t
            << " dt=" << dt << " V_max=" << V_max << "\n";
    }
}

if (rank == 0) std::cout << "SAFS smoke driver: " << step
    << " steps OK\n";

return 0;
}

```

3.2 Driver flow (high-level pseudocode)

```

void AssertSafsAttrs(const ParMesh& pmesh, const SafsTestParams& p) {
    const auto& bdr_attrs = pmesh.bdr_attributes;
    const auto& el_attrs = pmesh.attributes;
    const std::array<int, 6> expected_bdr = {p.attr_xm, p.attr_xp,
                                             p.attr_yp, p.attr_ym,
                                             p.attr_ztop, p.attr_zbot};

    // R-003: file-local helper to format Array<int> for diagnostic
    // messages. No global ListAttrs() exists in the SEAS codebase.
    auto fmt_attrs = [](const Array<int>& a) {
        std::ostringstream oss;
        for (int i = 0; i < a.Size(); ++i) {
            if (i > 0) { oss << ", "; }
            oss << a[i];
        }
        return oss.str();
    };

    for (int a : expected_bdr) {
        bool found = false;
        for (int i = 0; i < bdr_attrs.Size(); ++i) {
            if (bdr_attrs[i] == a) { found = true; break; }
        }
        MFEM_VERIFY(found,
            "SAFS mesh missing required boundary attribute " << a
            << " (mesh has [" << fmt_attrs(bdr_attrs) << "])");
    }

    bool has_domain = false;
    for (int i = 0; i < el_attrs.Size(); ++i) {
        if (el_attrs[i] == 10) { has_domain = true; break; }
    }
    MFEM_VERIFY(has_domain,

```

```

        "SAFS mesh missing element attribute 10 (bulk volume)");
// Note: tag 100 (fault interior) is NOT a boundary attribute – it is
// an interior face physical group. The fault-DOF count check comes
// later (after FaultGeometry construction).
}

```

3.3 AssertSafsAttrs body

3.4 Edge cases

- Rank 0 has no fault DOFs: legitimate when $np > 6$. FaultGeometry handles this (early-return path). The driver's CSV writes only on rank 0; per-rank $V_{\max}/\psi_{\max}$ are reduced via `mpi.GlobalReduce(MAX, ...)` before logging.
- `dt_init` exceeds `dt_max`: clamp via `std::min(...)`. Done in step 10.
- MUMPS unavailable in build: fallback to `SolverType::CG_AMG`. Print a warning. Detect via try-catch around `domain.Solve(...)`.
- `pmesh.bdr_attributes` does not include all expected: hard fail before any solve.

Acceptance Criteria

Local (laptop, `mfem-dev` env)

- ☐ `make seas_safs_test_driver` builds with no warnings.
- ☐ `make test` (existing seas unit-test suite) continues to pass — adding the new driver target must not break any existing test.
- ☐ No SIGSEGV / SIGFPE in any unit test.

The driver itself is not invoked end-to-end on the laptop. The ~1.1 M tet SAFS mesh exceeds practical local memory for a DG elasticity solve at $p=1$ with MUMPS factorization. Compile-readiness is the only local gate.

Frontera (driver end-to-end)

- ☐ `sbatch jobs/safs/safs_smoke_8N_400r_dev.sbatch` exits 0 on the development queue within the 2 hr walltime budget.
- ☐ CSV contains at least 1 accepted RK45 step (i.e. `step >= 1`).
- ☐ $V_{\max}$ per row stays in $[V_{\text{zero}}, 10 * V_p] \approx [1e-20, 1e-8]$ (no nucleation seed → no fast slip expected).
- ☐ $n_{\text{dt_rejects}}$ total $< 0.5 * n_{\text{steps}}$ (RK45 should accept most steps in this near-equilibrium regime).
- ☐ No NaN/Inf in any state column.

Phase 4 below describes the Frontera sbatch, the acceptance script, and the full set of CSV thresholds.

Dependencies

- Depends on: Phases 1 + 2.
- Required by: Phase 4 (Frontera run + acceptance check).

Phase 4: Frontera verification run and acceptance

Goal

A submittable Frontera sbatch job that runs the SAFS smoke driver end-to-end on the production cavity mesh, and a Python acceptance script that reads its CSV and reports PASS/FAIL on the thresholds below.

After this phase, the user has a one-line “is the SAFS mesh broken under SEAS DG?” check.

Files to Create

- `jobs/safs/safs_smoke_8N_400r_dev.sbatch` — Frontera sbatch script for the 8-node / 400-core / 2 hr development-queue run. Mirrors the conventions of `jobs/tpv102/tpv102_200m_p1_1.5s_50rank_dev.sbatch` (module loads, `LD_LIBRARY_PATH` setup, `cd /scratch2/...`, build guard for `MFEM_USE_PETSC` and `MFEM_USE_MUMPS`, build the new target, run with `ibrun`).
- `safs/code/check_safs_smoke.py` — Python script that ingests one CSV and prints PASS/FAIL on each acceptance metric below.

The plan does not add a laptop driver-run script. Local validation is compile + existing unit tests only (see Phase-3 acceptance).

Detailed Requirements

4.1 Frontera sbatch — `jobs/safs/safs_smoke_8N_400r_dev.sbatch` Required header:

```
#!/bin/bash
#SBATCH -J safs_smoke_8N_400r
#SBATCH -o safs_smoke_8N_400r_%j.out
#SBATCH -e safs_smoke_8N_400r_%j.err
#SBATCH -p development
#SBATCH -N 8
#SBATCH -n 400
#SBATCH -t 02:00:00
#SBATCH -A EAR20006
```

Body (mirrors `jobs/tpv102/.../50rank_dev.sbatch`):

```
export LC_ALL=C
export LANG=C

SBATCH_LOG_DIR="${SLURM_SUBMIT_DIR:-$PWD}"

module load intel/19.1.1
module load impi/19.0.9
module load hypre/2.31.0
module load mumps/5.3
module load parmetis
module load petsc/3.15
module load fftw3/3.3.8

export LD_LIBRARY_PATH="${TACC_HYPRE_LIB}:${TACC_PARMETIS_LIB}:\
${TACC_MUMPS_LIB}:${TACC_PETSC_LIB}:${TACC_FFTW3_LIB}:\
${LD_LIBRARY_PATH}"

cd /scratch2/10024/zhaochun/seas-project/seas-mfem

JOBS=8

grep -q '^MFEM_USE_PETSC *= YES' config/config.mk || \
{ echo "ERROR: no PETSc"; exit 1; }
grep -q '^MFEM_USE_MUMPS *= YES' config/config.mk || \
{ echo "ERROR: no MUMPS"; exit 1; }
```

```

cd miniapps/seas
make seas_safs_test_driver -j"${JOBS}" || \
    { echo "BUILD FAILED"; exit 1; }

RESULT_DIR="safs/code/results_8N_400r_job${SLURM_JOB_ID}"
mkdir -p "${RESULT_DIR}"

MESH=safs/mesh/output/newset_6_all6_garnetfirst/output/safs_newset_6_cavity.msh

echo "Git commit: $(git rev-parse HEAD)"
echo "Git branch: $(git rev-parse --abbrev-ref HEAD)"
echo "Output dir: ${RESULT_DIR}"
echo "Mesh: ${MESH}"

ibrun ./seas_safs_test_driver \
    --mesh "${MESH}" \
    --output-csv "${RESULT_DIR}/safs_smoke.csv" \
    --n-steps 100 \
    --order 1 \
    --solver mumps \
    --print-every 10 \
    > "${RESULT_DIR}/run.log" 2>&1
RC=$?

# Post-run acceptance check
cd "${SBATCH_LOG_DIR}"
python miniapps/seas/safs/code/check_safs_smoke.py \
    "${RESULT_DIR}/safs_smoke.csv" \
    > "${RESULT_DIR}/pass_check.txt"
CHECK_RC=$?

echo "driver_rc=${RC}, check_rc=${CHECK_RC}"
exit $(( RC | CHECK_RC ))

```

Notes on the choices: -N 8 -n 400 -t 02:00:00 -p development matches the user's stated budget (8 nodes, 400 cores total = 50 cores/node, 2 hr dev-queue). - The 400-core split is consistent with TPV102's 200m_p1_1.5s_400rank_*.sbatch scaling — Frontera Cascade Lake has 56 cores/node, so 50/node leaves headroom and gets us a clean 400 total. - --n-steps 100 is the same cap as used in Phase 4's pre-rewrite acceptance set. With $dt_{init} \approx 16$ days and $dt_{max} = 36$ days, 100 accepted steps cover roughly 5–10 simulated years — well past the 4-phase initialization transient. - make seas_safs_test_driver -j8 builds the new target only; this is additive over the existing seas Makefile and does not touch any existing target's recipe (per Phase-3 §3 "Files to Modify" guarantee). - The post-run check_safs_smoke.py exits non-zero if any metric fails; the sbatch returns the OR of driver and check exit codes so the slurm state field reflects either failure mode.

4.2 Acceptance metrics (encoded in **check_safs_smoke.py**) For the CSV produced by the Frontera run:

Metric	Threshold	Rationale
All t finite + monotone non-decreasing	exact	Time-stepper sanity
All V_max, V_min, psi_*, traction_max finite	exact	No NaN/Inf

Metric	Threshold	Rationale
$V_{\max}$ in $[V_{\text{zero}}, 10 \cdot V_p]$ for every row	physical	No nucleation seed -> V should not blow up
ψ in $[0, 5]$ for every row	physical	RSF state variable should not run away
$\text{traction}_{\max} < 100 \cdot \sigma_n$	$\sigma_n = 25$ MPa, bound 2.5 GPa	gross shear instability check
$n_{\text{dt_rejects total}} < 0.5 \cdot n_{\text{steps}}$	empirical	persistent rejection signals
slip_{l2} monotone	exact	stiffness blowup
non-decreasing across rows	exact	aging law forbids slip going backwards
Final $t > 0$ and step ≥ 1	exact	did at least one step succeed

Output: `pass_check.txt` with one PASS/FAIL line per metric and a final OVERALL: PASS or OVERALL: FAIL. Exit code 0 if ALL pass, else 1.

4.3 Local pre-flight (no driver run) Before submitting the sbatch, locally run:

```
cd miniapps/seas
make seas_safs_test_driver -j8      # build only
make test                          # existing unit tests
```

Both must succeed. Do not invoke `./seas_safs_test_driver` directly on a laptop — the SAFS mesh + MUMPS factorization will exhaust local memory long before the first time-step.

Acceptance Criteria

- ☐ `sbatch jobs/safs/safs_smoke_8N_400r_dev.sbatch` exits 0 within the 2 hr walltime.
- ☐ `pass_check.txt` reports OVERALL: PASS.
- ☐ `safs_smoke.csv` has at least 1 accepted RK45 step row.
- ☐ No NaN/Inf in any state column of the CSV.

Dependencies

- Depends on: Phase 3 (driver builds).
- Required by: nothing (this is the final mesh-shakedown gate).

Differences vs BP5 setup — quick-reference table

Aspect	BP5 reference	SAFS smoke test	Why
Mesh tag scheme	1=Natural, 3=Fault, 5=Dirichlet	1=xm, 2=xp, 3=yp, 4=ym, 5=ztop, 6=zbot, 100=fault	SAFS uses a different physical-group convention from <code>safs.geo</code>
Boundary condition mode	<code>BCMode::FarField</code> (attrs 1-4 Dir, 5-6 Nat)	Explicit <code>BoundaryConfig</code> (attrs 3,4 Dir; 1,2,5,6 Nat; fault=100)	User-requested: load only on y-faces; legacy enum can't represent this
Loading function	$u_X = \text{sgn}(Y) \cdot V_p \cdot t / 2$ on attrs 1-4	Same expression, but only attrs 3,4 see it	Same physical meaning; restricted footprint

Aspect	BP5 reference	SAFS smoke test	Why
Fault geometry ref_normal	Planar, $y=0$, $x=\text{along-strike}$, $z=\text{depth}$ (0,-1,0) — fault-normal	6 CFM components, complex 3D embedded (0,-1,0) (default; per-face orientation will not be globally consistent across the 6 faults but that's acceptable for a smoke test)	SAFS multi-fault realistic geometry One global ref_normal cannot match arbitrary fault normals; flag this in driver header comment
$a(x_2, x_3)$	VW core + transition + VS shell	uniform $a = a_0$ everywhere	“no VS patch, all VW”
$L(x_2, x_3)$	$L_{\text{nuc}}=0.13$ in patch, $L_0=0.14$ elsewhere	uniform $L_0 = 0.14$	“no VS patch” → no nucleation patch
V_{init}	$V_{\text{nuc}} = 0.01$ in nucleation, $V_{\text{init}} =$ V_p elsewhere	uniform $V_{\text{init}} = V_p$	No nucleation seed
τ_{pre}	uniform expression with $\delta\tau$ boost in nucleation	uniform (no $\delta\tau$)	Equilibrium initial state
Initial dt	$0.01 * L_{\text{nuc}} /$ $V_{\text{nuc}} = 1.3e-1$ s (fast)	$0.01 * L_0 / V_p =$ $1.4e6$ s, clamped to 0.1 yr (slow)	No nucleation → no fast initial transient
$t_{\text{final}} / n_{\text{steps}}$	1800 yr / millions of steps	1 yr / 100 steps (CLI cap)	Smoke test
Output	SCEC station time series + checkpoints	One CSV with per-step max/min state	No benchmark validation needed
Validator (mesh-side)	N/A	The mesh validate_msh.py already reports 12/13 PASS; failing check is check_10 R-402 near-fault sliver count, accepted (intrinsic to SAFS branching geometry); topology checks 5/12/13 PASS	Mesh quality is independent of physics

Testing Strategy

Two-tier: local gates the build, Frontera gates the physics.

Local (laptop, **mfem-dev** env)

- Phase 1 unit checks — header-only, hand-rolled assertions for `MakeSafsBoundaryConfig` and `SafsTestParams::Validate()`. Compiled into `seas_safs_test_driver` but executed at startup only. (No separate make target.)
- Phase 2 unit checks — assertions for `bp5.a_of_x2_x3(...)`, `bp5.IsNucleationZone(...)`, `bp5.L_of_x2_x3(...)` after `OverrideToUniformVW`. Same form: in `main()` startup, before any solve.
- Build gate — `make seas_safs_test_driver` builds with no warnings; `make test` (existing seas test suite) continues to pass.

The local gate stops at “binary exists and unit tests pass”. We do NOT invoke the driver end-to-end locally — the SAFS mesh exceeds laptop memory for a DG MUMPS factorization.

Frontera (driver end-to-end)

- Phase 4 acceptance — `check_safs_smoke.py` against the CSV produced by `jobs/safs/safs_smoke_8N_400r_de`
Pass = all thresholds in §4.2 satisfied; written to `pass_check.txt`. The sbatch returns the OR of driver and check exit codes so a slurm monitor can flag either failure mode.

No new C++ unit-test executable is added; the local assertions are header-only inside the driver and the Frontera acceptance is a Python script. New surface area: 1 driver .cpp, 2 headers, 1 sbatch, 1 Python script.

Risk Assessment

Risk	Likelihood	Detection	Mitigation
The ~270 near-fault sliver tets cause MUMPS to produce ill-conditioned solves	medium	--check-residual reports $ Kx-b / b > 1e-8$ in the first solve	Switch to <code>SolverType::CG_AMG</code> and document; if still ill-conditioned, that's the canary the user wanted (decide whether to invest in further sliver removal — e.g. tighter <code>L0-CAL_CAVITY_GAMMA_THRESH</code> or a rewrite of <code>mmg3d_local_patch</code>)
Fault basis with single <code>ref_normal = (0,-1,0)</code> produces left-handed frames on faults whose true normal is in +y	high (fact of geometry)	<code>FaultBasis::Compute()</code> flips signs internally; loading on y-faces drives slip in some direction on every fault but the SIGN may differ across faults	Acceptable for mesh-shakedown: we are NOT validating sense-of-slip. Document in driver header. For physics validation, a future plan needs per-fault <code>ref_normal</code>
Fault interior face mapping fails because tag 100 has 5398 triangles split across 6 disconnected components	low	<code>FaultGeometry::NumGlobalFaces</code> is much smaller than <code>5398 * nbf_per_face</code>	hard-fail in driver, user re-runs mesh pipeline
Friction solver (Brent) hits log domain because $V_{init} = V_{zero} \approx 1e-20$ on the dip component	low (BP5 has same setup)	Brent's bracket guard returns frictionless limit	The existing BP5 friction solver handles this case (debug v7)

Risk	Likelihood	Detection	Mitigation
Steiner clearance (free-surface 100m) interacts with the BP5 dirichlet expression's 1km dead-band	low	DOFs at $\ y\ < 1000\text{m}$ get full $Vp \cdot t$ instead of $0.5 \cdot Vp \cdot t$. SAFS box is at $\ y\ \geq 130\text{km}$, so no DOF is in the dead-band	Verified by domain box (Phase 4 check); flag in driver if any boundary DOF has $\ y\ < 1000\text{m}$
BP5Params::Validate() rejects our overrides	medium	Validate() throws if $a_{\text{max}} > b$ fails	Driver does NOT call <code>bp5.Validate()</code> — it calls <code>params.Validate()</code> (the SAFS one)
Per-DOF IsNucleationZone false-positive on a numerical-eps boundary	low	<code>nucleation_eps = 0.0</code> after override	Verified in unit assertion in Phase 2
Time stepper diverges in first step due to non-equilibrium initial state on multi-fault	low	aborts on $V_{\text{max}} > 10\text{ m/s}$ (step diagnostic)	The init is constructed to be equilibrium by Tandem-style psi solve; if it still blows up, that's the canary

What this plan deliberately leaves out

- Per-fault diagnostics across the 6 CFM components. Pooling them into one fault for shakedown. A separate plan can split them later using `fault_provenance.json`.
- Restart / checkpoint. 100 steps doesn't need it.
- Visualization of the slip field. Driver writes CSV only; user can inspect the existing `.vtu` of the mesh for the fault geometry.
- Convergence study (p-refinement, h-refinement). Out of scope.
- Local end-to-end driver run. Local laptops cannot factor the $\sim 1.1\text{ M}$ tet SAFS DG matrix; the driver runs on Frontera only (Phase 4). Locally we gate on compile + existing unit tests.
- Frontera production-queue / multi-node-scaling sweep. This plan is dev-queue only (8 nodes / 400 cores / 2 hr). A separate plan can do scaling once this single-config shakedown passes.